1) Ontological Engineering

- Representing abstract concepts is called as Ontological Engineering.
- **Ontological Engineering** is a field within artificial intelligence and knowledge representation that focuses on the creation, representation, and management of ontologies.
- An ontology is a formal representation of a set of **concepts** within a **domain** and the **relationships** between those **concepts**. This framework is essential for facilitating understanding and sharing knowledge in various AI applications.

It describes:

- What kinds of objects exist
- What properties they have
- How they relate to one another

Ontology = *Vocabulary* + *Meaning* of terms

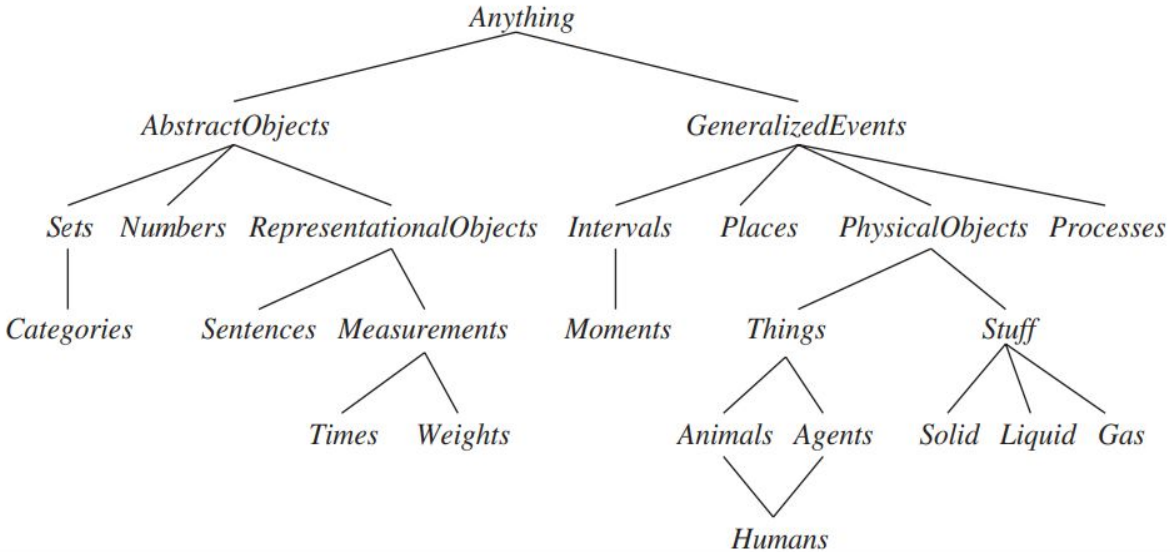


Figure: The upper ontology of the world, showing the topics. Each link indicates that the lower concept is a specialization of the upper one. Specializations are not necessarily disjoint—a human is both an animal and an agent.

Why Ontological engineering

- Ontological engineering plays a crucial role in structuring knowledge for AI applications.
- By using first-order logic to formally represent the relationships and rules governing a domain, systems can reason about the information effectively.
- This structured approach enhances the ability of AI to understand, infer, and act based on the knowledge contained within ontologies.

Key **Components** of Ontological Engineering

- **Concepts**: These are the entities or classes that exist within a domain. For example, in a *medical* ontology, concepts could include "Disease," "Symptom," and "Treatment."
- **Relationships**: These define how concepts are related to one another. In our medical ontology, a relationship could be "*hasSymptom*," connecting "*Disease*" to "*Symptom*."
- **Attributes**: These provide additional information about concepts. For instance, the "Disease" concept might have attributes like "*name*," "*severity*," and "*contagiousness*."

- **Domain: Medical Ontology**
- **Concepts**:
  - Disease
  - Symptom
  - Treatment
- **Relationships**:
  - ***"hasSymptom"***
    (e.g., "Flu hasSymptom Fever")
  - ***"treatedBy"***
    (e.g., "Flu treatedBy Antiviral Medication")

Components of an Ontology

- **Classes (Concepts)**
  – Categories of objects (e.g., Animal, Vehicle)
- **Instances (Individuals)**
  – Specific objects (e.g., Tiger, Honda-City)
- **Attributes (Properties)**
  – Describe classes/individuals
  (e.g., Color, Age)
- **Relationships (Relations)**
  – Define how concepts are related
  (e.g., Eats, PartOf, LocatedIn)
- **Axioms**
  – Logical rules and constraints

Example

A simple ontology for Animals :

Classes : Animal, Mammal, Bird

Relationships : IsA(Mammal, Animal), HasProperty(Animal, Eats)

Instances : Tiger → Mammal, Parrot → Bird

Axioms : Mammals are warm-blooded, Birds can fly (usually)

2) Categories and Objects

In knowledge representation, **categories** help organize **objects** into structured groups based on shared properties.
This allows an intelligent agent to simplify reasoning by dealing with **classes** of things instead of individual instances.

Objects are individuals, Categories are groups of similar objects.

- An object is a **member** of a category.

$$BB_9 \in Basketballs$$

- A category is a **subclass** of another category.

$$Basketballs \subset Balls$$

- All members of a category have some **properties**.

$$(x \in Basketballs) \Rightarrow Spherical(x)$$

- Members of a category can be recognized by some properties.

$$Orange(x) \wedge Round(x) \wedge Diameter(x) = 9.5\ inch \wedge x \in Balls \Rightarrow x \in Basketballs$$

- A category as a whole has some properties.

$$Dogs \in DomesticatedSpecies$$

Disjoint Categories

Two (or more) categories are **disjoint** if **no object can belong to both categories at the same time**.

If A and B are disjoint : $A \cap B = \varnothing$

Example : Cat , Dog

No individual is both a cat and a dog.

Formally : $\forall x : Cat(x) \rightarrow \neg Dog(x)$

Exhaustive Decomposition

A category is **exhaustively decomposed** when all its members must belong to **at least one** subcategory. If C is decomposed into A, B, and D, then, $C = A \cup B \cup D$

No instance of C falls outside the listed subcategories.

Example : Category : Student
Subcategories : Undergraduate, Graduate

Every student must be either undergraduate or graduate.

Formally : $\forall x : Student(x) \rightarrow (Undergrad(x) \vee Graduate(x))$

3) Partition

A **partition** is formed when subcategories are **both disjoint** and an **exhaustive decomposition** of the parent category. Partition = Disjoint + Exhaustive

So for subcategories A, B, C of category D :

1. $A \cap B \cap C = \varnothing$ (pairwise disjoint)

2. $A \cup B \cup C = D$ (exhaustive)

Example : Category: Traffic light
Subcategories: Red, Yellow, Green

- A light must be exactly one of these (exhaustive)
- It cannot be more than one simultaneously (disjoint)

This set forms a **partition**.

Formally : $\forall x : Light(x) \rightarrow (Red(x) \oplus Yellow(x) \oplus Green(x))$ ($\oplus$ = exclusive OR)

Physical Composition :

It refers to the **part–whole relationship** among objects. It helps represent how complex objects are constructed from simpler components. A physical object can be defined as a composition of its constituent parts.

Measurements :

Objects often have **quantitative attributes** that can be numerically described. These attributes are called **measurements**. Values belong to **scales** such as:

- Length
- Weight
- Temperature
- Speed

3) Events

In knowledge representation, **events** are entities that represent **things that happen in time**. They describe **dynamic aspects** of the world; changes in states rather than static properties.

Events = Temporally-located occurrences that may modify the world.

Examples :

- A ball falling
- A meeting starting
- A door opening
- A robot picking up an object

In contrast to static objects (car, person), **events describe actions and changes**.

Why events are needed ?

Real-world systems are not static. To reason realistically, an AI must represent:

- What happened
- When it happened
- What it affected

This is crucial for:

- Planning
- Causal reasoning
- Temporal reasoning
- Story understanding
- Robotics

Knowledge Representation → Ontologies and Categories → Events & Time

The complete set of predicates for one version of the event calculus.

i.	$T(f, t1, t2)$	Fluent f is true for all times between $t1$ *and* $t2$
ii.	$Happens(e, t1, t2)$	Event e starts at time $t1$ and ends at $t2$
iii.	$Initiates(e, f, t)$	Event e causes fluent f to become true at time t
iv.	$Terminates(e, f, t)$	Event e causes fluent f to cease to be true at time t
v.	$Initiated(f, t1, t2)$	Fluent f become true at some point between $t1$ and $t2$
vi.	$Terminated(f, t1, t2)$	Fluent f cease to be true at some point between $t1$ and $t2$
vii.	$t1 < t2$	Time point $t1$ occurs before time $t2$

$$E = Flyings(a, here, there) \text{ and } Happens(E, t1, t2)$$
$$\Rightarrow \quad Terminates(E, At(a, here), t1) \wedge Initiates(E, At(a, there), t2)$$

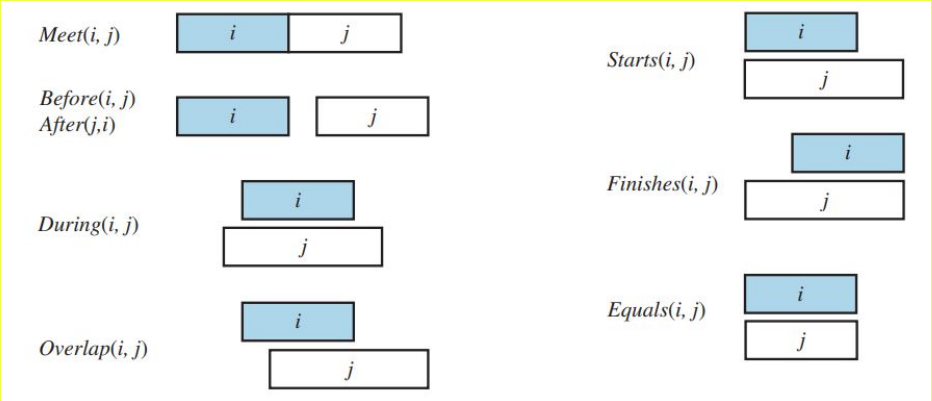
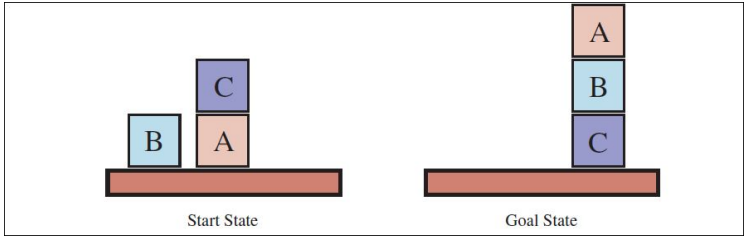


Figure : Predicates on time intervals

Block-world problem



$Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A)$
$\quad \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C) \wedge Clear(Table))$
$Goal(On(A, B) \wedge On(B, C))$
$Action(Move(b, x, y),$
$\quad$ PRECOND: $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$
$\quad\quad (b \neq x) \wedge (b \neq y) \wedge (x \neq y),$
$\quad$ EFFECT: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$
$Action(MoveToTable(b, x),$
$\quad$ PRECOND: $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge Block(x),$
$\quad$ EFFECT: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$

Figure 11.4 A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.

The complete set of predicates for one version of the event calculus.

i.	$T(f, t1, t2)$	Fluent f is true for all times between $t1$ and $t2$
ii.	$Happens(e, t1, t2)$	Event e starts at time $t1$ and ends at $t2$
iii.	$Initiates(e, f, t)$	Event e causes fluent f to become true at time t
iv.	$Terminates(e, f, t)$	Event e causes fluent f to cease to be true at time t
v.	$Initiated(f, t1, t2)$	Fluent f become true at some point between $t1$ and $t2$
vi.	$Terminated(f, t1, t2)$	Fluent f cease to be true at some point between $t1$ and $t2$
vii.	$t1 < t2$	Time point $t1$ occurs before time $t2$

$$E = Flyings(a, here, there)\ and\ Happens(E, t1, t2)$$
$$\Rightarrow \quad Terminates(E, At(a, here), t1) \wedge Initiates(E, At(a, there), t2)$$

4) Mental Events

Mental events refer to the occurrences that happen within a cognitive system, whether that system is a human mind or an AI model.

These events can include:

1. Processing Information
2. Decision Making
3. Learning

Mental Objects-

Mental objects in AI refer to the representations of knowledge, data, or entities that the system manipulates during its operations.

These can include:

1. Data Structures
2. Knowledge Representations
3. Agents

- The relationship between mental events and mental objects is integral to the functioning of AI systems. For example:
- **Perception** (a mental event) involves interpreting data inputs (mental objects) to identify features or patterns.
- **Reasoning** often operates on the representations of knowledge (mental objects) to draw conclusions or make predictions based on new information (mental events).
- **Learning** processes (mental events) adjust the parameters or structures of models (mental objects) based on feedback from the environment.

Example-

- Ajay knows superman can fly.
- knows(Ajay, CanFly(Superman))
- If Vijay is superman then, we must conclude Ajay knows Vijay can fly.
- (Superman=Vijay) ^ Knows(Ajay, CanFly(Superman)) = knows(Ajay, CanFly(Vijay))
- This property is called **referential transparency**.

Knowledge about Knowledge

- Knowledge about the knowledge of other agents is important.
- In order for us to use inference in the best possible way we must construct a knowledge base about our knowledge base.

Knowledge vs Belief

- Knowledge in terms of AI is something which is always true.
- Belief deals with probability.

- In **referentially opaque** we cannot swap 2 equal terms , opposite to referential transparency.
- Why this important?
- Just because Ajay beliefs Superman can fly doesn't mean he beliefs Vijay can fly.
- By making Flies(Superman) opaque, we can say Beliefs(Ajay, Flies(Superman)) without saying Beliefs(Ajay, Flies(Vijay)) .

5) Automated Planning

Automated Planning is the branch of AI concerned with **generating a sequence of actions (a plan) that transforms the current state of the world into a desired goal state.** It answers: **“What should the agent do next?”**
to achieve its objective **without** requiring a predefined action script.

Planning is different from simple search because actions have:

- Preconditions
- Effects
- Temporal structure

Planning allows **deliberative decision-making**, agents think before acting.

Why Planning is Needed ?

An intelligent agent must:

- Decide how to achieve goals
- Anticipate future states
- Avoid trial-and-error behavior

Used in : Robotics, Logistics, Scheduling, Game AI, Autonomous vehicles, Space missions

Components of a Planning System

Planning is defined over Action**,** described by :

- Preconditions: must be true to apply action
- Effects: facts that change after action

Component	Meaning
Initial State	Current description of the world
Goal State(s)	Desired outcome conditions
Actions (Operators)	Defined by Preconditions & Effects
Plan	Sequence of actions to reach the goal

Action : PickUp(x)
Precondition : Clear(x) ∧ HandEmpty
Effect : Holding(x) ∧ ¬Clear(x)

6) Classical Planning

Classical Planning is a simplified model of planning used when the environment satisfies certain assumptions.

It assumes:

1. **Fully observable** : agent knows the complete current state
2. **Deterministic actions** : actions have predictable effects
3. **Static world** : world does not change unless the agent acts
4. **Discrete** : states and actions are finite
5. **Single agent** : no multi-agent interference

Goal : Find a **sequence of actions** that transforms the initial state into a goal state.

This is the most studied planning model in AI.

Representation

State : Sets of facts describing the world (e.g., At(Robot, RoomA), BoxIn(RoomA))

Actions (Operators)

Defined by:

- Preconditions
- Effects (add & delete lists)

Action : PickUp(x)
Pre : Clear(x) ∧ HandEmpty
Add : Holding(x)
Del : Clear(x)

Goal : A set of conditions to satisfy.

A plan is a sequence:

a1 → a2 → … → an

7) Algorithms for Classical Planning

State-Space Search :

A) Forward Search (Progression)

Start → Goal

- Begin at the initial state
- Apply actions whose preconditions are satisfied
- Generate successor states
- Search until a goal is reached

Properties:

- Easily guided by heuristics
- May generate many irrelevant states

Good for problems where initial state is simple.

B) Backward Search (Regression)

Goal → Start

- Start from goal conditions
- Regress through actions that can achieve goals
- Produce subgoals until initial state is reached

Advantages:

- Avoids exploring irrelevant states
- Works well when goal is small

PDDL : Planning Domain Definition Language

PDDL (Planning Domain Definition Language) is a **standard language to describe planning problems** used by classical planning systems.

It was introduced to:

- Provide a **common representation format** for planners
- Allow comparison of planning algorithms on benchmark problems
- Separate **domain knowledge** from **problem instances**

Used in :

- STRIPS-style planning
- Classical planning competitions

Basic Structure :

PDDL divides planning into:

Domain : Defines general rules, containing : Requirements, Predicates, Actions (operators)

Problem : Defines specific instance, containing : Objects, Initial state, Goal state

8) Forward State-Space Search (Progression Planning)

Forward state-space search starts from the **initial state** and applies actions to generate new states until the **goal state** is reached. It progresses from **Initial State → Goal State**

Also called:

- **Progression search**
- **Forward chaining (in planning context)**

Process

1. Start with the initial state S0
2. Apply all applicable actions (whose preconditions are true in S0)
3. Generate successor states
4. Continue applying actions to new states
5. Stop when a state satisfies the goal condition

Example

Suppose:
Goal : `(on A B)`
Initial : `(ontable A), (ontable B), (clear A), (clear B), (handempty)`

Forward search begins by applying valid actions from the start, e.g.

- pick-up(A)
- pick-up(B)

etc., leading eventually to `(on A B)`.

Advantages

- Natural direction (realistic modeling)
- Avoids need to work backward
- Easy to simulate exact world states
- Works well with complete initial information

Disadvantages

- Very large branching factor
- May consider many irrelevant actions
- Hard to guide search unless strong heuristics are used

9) Backward State-Space Search (Regression Planning)

Backward state-space search starts from the **goal condition** and reasons backward to the **initial state** by regressing goals over actions. It proceeds **Goal State → Initial State**

Also called **Regression search**.

Process

1. Start with the goal condition G
2. Select an action that can achieve a part of G
3. Regress the goal through that action
4. This produces a new sub-goal
5. Continue until sub-goal is satisfied by the initial state

Instead of applying actions to states, we apply **goal regression over actions**.

Regression:

> Replace satisfied goal conditions by the preconditions needed to achieve them.

Example

Goal : `(on A B)`

Action : `(stack A B)`

- Effect: `(on A B)`
- Preconditions:
  - `(holding A)`
  - `(clear B)`

So we substitute goal : `G' = (holding A) ∧ (clear B)`

Continue regression until we reach a condition true in initial state.

Advantages

- Lower branching factor, fewer actions are relevant
- Focuses only on relevant actions
- Efficient for conjunctive goals

Disadvantages

- More complex implementation
- Hard when goals include negative information
- Hard for conditional effects
- Very difficult when domain is not easily invertible

BLOCKS WORLD : Domain Overview

Predicates

- `on(x, y)` – block x is on block y
- `ontable(x)` – block x is on the table
- `clear(x)` – nothing on x
- `handempty` – agent not holding anything
- `holding(x)` – agent holding block x

Actions

1. **pick-up(x)**
2. **put-down(x)**
3. **stack(x, y)**
4. **unstack(x, y)**

Goal state : Make A on B `(on A B)`

Initial state : `ontable(A), ontable(B), clear(A), clear(B), handempty`

FORWARD STATE-SPACE SEARCH (PROGRESSION)

Initial State S0 : ontable(A), ontable(B), clear(A), clear(B), handempty

Step 1 : Applicable actions : pick-up(A), pick-up(B)

Let's choose : `pick-up(A)`

Effects:

```
holding(A)
not ontable(A)
not handempty
```

State S1

```
holding(A)
ontable(B)
clear(B)
not ontable(A)
not handempty
```

Step 2 : Applicable action : `stack(A, B)`

Preconditions : `holding(A), clear(B)`

- on(A, B)
- clear(A)
- handempty

State S2

```
on(A,B)
clear(A)
handempty
ontable(B)
```

Goal satisfied → STOP

Forward Search Path : `S0 → pick-up(A) → S1 → stack(A, B) → S2 (Goal)`

BACKWARD STATE-SPACE SEARCH (REGRESSION)

Step 1 : Regress

Action : `stack(A, B)`
Effects include : `(on A B)`

Preconditions : `holding(A), clear(B)`

So after regression, new sub-goal G1 becomes:

```
holding(A)
clear(B)
```

Step 2

How to achieve `holding(A)`?
Action : `pick-up(A)`

Preconditions : `clear(A),ontable(A), handempty`

So new sub-goal G2:

```
clear(A)
ontable(A)
handempty
clear(B)
```

Step 3 : Does G2 match initial state?

Initial state:

```
ontable(A)
ontable(B)
clear(A)
clear(B)
handempty
```

Yes, All goals satisfied. So, regression succeeded.

Backward Search Path :

Reverse action order:

1. pick-up(A)
2. stack(A, B)

```
Goal(on A B)
       ^
       | regress(stack(A,B))
       |
Subgoal: holding(A), clear(B)
       ^
       | regress(pick-up(A))
       |
Initial satisfied → STOP
```